VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Faculty of Computer Science and Engineering

Textbook Exercises: Pipelining

Advanced System Architectures

CO5059

Instructor: Tran Ngoc Thinh

Notes by: Phan Nhat Minh (minh.phannhat95@hcmut.edu.vn)

Semester: 261

Hennessy & Patterson, Computer Architecture: A Quantitative Approach, *5th ed., Appendix C (Pipelining: Basic and Intermediate Concepts) exercises C.1–C.14. A map of every exercise with its page and relevance to Lecture 3, worked solutions for the ones our lectures cover, and hints for the rest.*

Last updated: October 4, 2026

Contents

1 How to Use This File **1**

2 Exercise Map **1**

3 Worked Solutions: High Priority **1**
Exercise C.1 1
Exercise C.2 2
Exercise C.3 3
Exercise C.4 3
Exercise C.7 3

4 Hints: Medium Priority **4**
Exercise C.5 4
Exercise C.6 4

1 How to Use This File

- The exercise text is **not** copied here – open the book (`references/Computer Architecture - A Quantitative Approach.pdf`) at the page given. "Book p." is the printed page (Appendix C pages are numbered C-1, C-2, …); "PDF p." is the page in the PDF viewer.
- The book's pipeline is the same 5-stage MIPS pipeline as our lecture (it writes 64-bit opcodes such as `LD`, `DADDI`, `BNEZ`).
- Unless an exercise says otherwise, the solutions assume what the book does in Section C.2: registers are written in the first half and read in the second half of a cycle, and branches are resolved in **ID**, so "flushing" costs one cycle (Fig. C.11). In diagrams, `stl` means a stall cycle.

2 Exercise Map

Ex.	Book p.	PDF p.	Topic	Priority
C.1	C-82/83	729–730	Loop: dependences, timing without/with forwarding, branch schemes, 10-stage pipeline, CPI	**High**
C.2	C-83	730	Branch penalties in a 4-deep and a 15-deep pipeline	**High**
C.3	C-83	730	Unbalanced stages: clock, CPI with stalls, speedup	**High**
C.4	C-84	731	Branch resolved in ID creates a data hazard	**High**
C.5	C-84/85	731–732	Pipeline for a register-memory ISA: adders, ports, forwarding, hazards	Medium
C.6	C-85/86	732–733	Adding register-memory ALU ops to the 5-stage pipeline	Medium
C.7	C-86	733	Deeper pipeline: faster clock vs. more stalls	**High**
C.8–C.11	C-86	733	R4000 forwarding/hazard tables; FP pipeline tables	Skip (later topic)
C.12–C.13	C-86/88	733–735	DAXPY on FP pipeline; scoreboarding	Skip (later topic)
C.14	C-88	735	Multicycle FP units: structural and WAW hazards	Skip (later topic)

"Later topic": multicycle FP pipelines, the R4000 and scoreboarding (Sections C.5–C.7) go beyond our Lecture 3; they belong with the instruction-level-parallelism lectures.

3 Worked Solutions: High Priority

Exercise C.1 (book p. C-82/83, PDF p. 729–730)

Asks: for the loop `LD R1; DADDI R1; SD R1; DADDI R2; DSUB R4; BNEZ R4` (99 iterations, since R3 = R2 + 396 and R2 grows by 4): list the dependences; time the loop without forwarding (flush on branch), with forwarding (predict not taken / predict taken), and on a 10-stage pipeline; then clock cycles and CPI.

Solution. **(a)** RAW dependences: R1 from `LD` to `DADDI R1`; R1 from `DADDI R1` to `SD`; R2

from `DADDI R2` to `DSUB`; R4 from `DSUB` to `BNEZ`; and the loop-carried R2 from `DADDI R2` to the next iteration's `LD` and `SD`. (Also WAR: `LD`/`SD` read R2 before `DADDI R2` writes it.)

(b) No forwarding; a consumer's ID can be no earlier than the producer's WB:

```
                c1  c2  c3  c4  c5  c6  c7  c8  c9  c10 c11 c12 c13 c14 c15 c16 c17 c18
LD    R1,0(R2)  IF  ID  EX  MEM WB
DADDI R1,R1,#1      IF  ID  stl stl EX  MEM WB
SD    R1,0(R2)          IF  stl stl ID  stl stl EX  MEM WB
DADDI R2,R2,#4              IF  stl stl ID  EX  MEM WB
DSUB  R4,R3,R2                      IF  ID  stl stl EX  MEM WB
BNEZ  R4,Loop                           IF  stl stl ID  stl stl EX  MEM WB
LD (next iter.)                                                 IF
```

The branch is resolved in ID in cycle 15, so the next iteration's `LD` is fetched in cycle 16: 15 cycles per iteration. Total $= 98 \times 15 + 18 = 1488$ cycles.

(c) Full forwarding, predict not taken. Only the load-use pair stalls, plus one stall because `BNEZ` needs R4 in ID but `DSUB` produces it at the end of EX (exactly the problem of C.4):

```
                c1  c2  c3  c4  c5  c6  c7  c8  c9  c10 c11 c12 c13 c14
LD    R1,0(R2)  IF  ID  EX  MEM WB
DADDI R1,R1,#1      IF  ID  stl EX  MEM WB
SD    R1,0(R2)          IF  stl ID  EX  MEM WB
DADDI R2,R2,#4              IF  ID  EX  MEM WB
DSUB  R4,R3,R2                  IF  ID  EX  MEM WB
BNEZ  R4,Loop                       IF  ID  stl EX  MEM WB
LD (next iter.)                             IF  ID  EX  MEM WB
```

`BNEZ` resolves in cycle 9; the predicted fall-through instruction is squashed and the target `LD` is fetched in cycle 10: 9 cycles per iteration. Total $= 98 \times 9 + 12 = 894$ cycles.

(d) Predict taken gives the same 894 cycles: in this pipeline the target address is also computed in ID, so a taken branch cannot be fetched any earlier (as noted in our lecture).

(e) 10 stages (IF1 IF2 ID1 ID2 EX1 EX2 M1 M2 WB1 WB2), forwarding from the end of a stage pair to the start of the pair that needs it. If instruction p starts at cycle s_p, a dependent ALU instruction must start at $s_p + 2$ or later, and a dependent use of a load at $s_p + 4$ or later. Assuming the branch target is known after ID2 and the condition is tested in EX1: `LD` 0, `DADDI` 4, `SD` 5, `DADDI R2` 6, `DSUB` 8, `BNEZ` 10, next `LD` at 14. That is 14 cycles per iteration, about $98 \times 14 + 20 = 1392$ cycles in total. (Different assumptions about where the branch resolves change this number; state yours.)

(f) 5-stage: $0.8 + 0.1 = 0.9$ ns. 10-stage: $0.4 + 0.1 = 0.5$ ns.

(g) 5-stage: 9 cycles per 6 instructions, CPI $= 1.5$, average time per instruction $= 1.5 \times 0.9 = 1.35$ ns. 10-stage: $14/6 = 2.33$ cycles, $2.33 \times 0.5 = 1.17$ ns. The deeper pipeline wins only modestly: its clock is 1.8× faster but its CPI is 1.56× worse. □

Exercise C.2 (book p. C-83, PDF p. 730)

Asks: with 15% conditional branches (60% taken) and 1% jumps, how much faster the machine would be without branch hazards, for a 4-deep and a 15-deep pipeline.

Solution. If a branch is resolved at the end of stage r and only stage 1 can proceed regardless, the penalty is $r - 1$ cycles.

(a) Jumps: $2 - 1 = 1$; conditional: $3 - 1 = 2$. CPI $= 1 + 0.01 \times 1 + 0.15 \times 2 = 1.31$, so 1.31× faster without hazards.

(b) Jumps: 4; conditional: 9. CPI $= 1 + 0.01 \times 4 + 0.15 \times 9 = 2.39$, so 2.39× faster.

Deep pipelines make branches dramatically more expensive – the motivation for good branch prediction. (If you assume only taken branches pay, the numbers become $1 + 0.01 + 0.15 \times 0.6 \times 2 = 1.19$ and $1 + 0.04 + 0.15 \times 0.6 \times 9 = 1.85$; say which you assume.) □

Exercise C.3 (book p. C-83, PDF p. 730)

Asks: a 7 ns single-cycle machine split into stages of 1, 1.5, 1, 2, 1.5 ns with 0.1 ns register delay: pipelined clock, CPI with a stall every 4 instructions, speedup, and speedup with infinitely many stages.

Solution.

(a) $2 + 0.1 = 2.1$ ns (slowest stage plus register).

(b) CPI $= 1 + 1/4 = 1.25$.

(c) $7/(1.25 \times 2.1) = 7/2.625 = 2.67$.

(d) With infinitely many stages the logic per stage goes to 0 and the clock approaches the 0.1 ns register delay: speedup $\to 7/0.1 = 70$ (or $7/(1.25 \times 0.1) = 56$ if the 1-in-4 stalls remain). Register overhead is the limit on how deep a pipeline is worth making.

□

Exercise C.4 (book p. C-84, PDF p. 731)

Asks: a code snippet where resolving the branch in ID causes a data hazard even with forwarding.

Solution.
```
DADD  R1, R2, R3
BEQZ  R1, Target
```

`BEQZ` needs R1 in its ID stage (cycle 3), but `DADD` produces it only at the end of its EX (cycle 3): one stall even with forwarding. A load is worse: `LD R1,0(R2); BEQZ R1,Target` needs 2 stalls, because the data exists only at the end of MEM. Moving the branch earlier saves branch cycles but creates these new data stalls. □

Exercise C.7 (book p. C-86, PDF p. 733)

Asks: 5-stage pipeline (1 ns, a data stall every 5 instructions) vs. 12-stage (0.6 ns, 3 stalls every 8 instructions); 20% branches with 5% mispredicted, penalties 2 and 5 cycles.

Solution.

(a) Data hazards only: $\text{CPI}_5 = 1 + 1/5 = 1.2$, $\text{CPI}_{12} = 1 + 3/8 = 1.375$. Time per instruction: $1.2 \times 1 = 1.2$ ns vs. $1.375 \times 0.6 = 0.825$ ns. Speedup $= 1.2/0.825 = 1.45$.

(b) Mispredictions per instruction $= 0.2 \times 0.05 = 0.01$: $\text{CPI}_5 = 1.2 + 0.01 \times 2 = 1.22$, $\text{CPI}_{12} = 1.375 + 0.01 \times 5 = 1.425$. (Speedup is now $1.22/(1.425 \times 0.6) = 1.43$.)

□

4 Hints: Medium Priority

Exercise C.5 (book p. C-84/85, PDF p. 731–732)

Asks: a 6-stage pipeline (IF RF ALU1 MEM ALU2 WB) for a register-memory ISA: count adders and register/memory ports, design the forwarding, list remaining hazards and control hazards.

Solution. Walk an instruction mix through the stages cycle by cycle and look for the worst case in a single cycle:

- Adders: PC increment (IF), address/branch-target add (ALU1), operation (ALU2) – several can be busy at once.
- Ports: RF reads up to 2 (or 3 for a store with a register-memory form), WB writes 1, memory: 1 instruction read and 1 data access per cycle.
- Forwarding: results appear at the end of ALU2 or MEM and are needed at the start of ALU1 (addresses) or ALU2 (operands); consider forwarding across one intervening instruction too.
- A result needed for an address in ALU1 one instruction later cannot be forwarded in time: those become stalls (address-generation interlocks). Branches compare in ALU2, so the branch penalty is several cycles.

□

Exercise C.6 (book p. C-85/86, PDF p. 732–733)

Asks: reorder the 5 stages so ALU operations can take a memory operand addressed by a register only; new forwarding paths, new hazards, and effects on instruction count and CPI.

Solution. Put MEM before EX (IF ID MEM EX WB): with register-indirect addressing no address calculation is needed, so memory can be read before the ALU. New paths: forward results to the MEM stage (addresses) as well as to EX. New hazard: an ALU result used as an address by the next instruction must wait (the address is needed in MEM, which now comes *before* EX). Instruction count drops when a load+ALU pair becomes one instruction, but rises when displacement addressing must be replaced by an explicit add. CPI changes through these new address hazards and the removed load-use stalls. □